

FINAL YEAR ENGINEERING PROJECT REVIEW & EVALUATION

Department of Computer Science & Engineering / Cloud Computing

OFFICIAL EVALUATOR & DEMONSTRATION GUIDE

Workload-Forecasting Driven Proactive Pod Scheduling for Kubernetes Clusters

Using LSTM-XGBoost Ensemble With Multi-Objective Node Ranking

Project Domain:	Cloud Computing, Distributed Systems & Machine Learning
Primary Innovation:	Zero-Lag Proactive Scheduling bypassing Reactive HPA cold starts
Core ML Architecture:	Hybrid Temporal Ensemble (TensorFlow LSTM + XGBoost Regressor)
Evaluation Results:	0.00% SLO Violations, -84.8% P95 Latency (735ms → 111ms)
Demonstration Platform:	Interactive Live Portal (Localhost + Netlify Cloud Shell Console)
Target Cluster:	Kubernetes v1.29 / Google Kubernetes Engine (GKE Standard)

Candidate Presentation Guide & Technical Defense Script • Confidential Evaluation Material

1. Executive Summary & Research Contribution

✓ The Core Problem Solved

Traditional Kubernetes schedulers and the Horizontal Pod Autoscaler (HPA) are strictly **reactive**. They trigger scaling only after worker nodes cross high CPU/memory utilization thresholds (e.g., 80%). Because spinning up containerized pods, scheduling, pulling images, and starting runtimes takes **30 to 90 seconds**, sudden traffic surges cause severe **cold-start latency spikes**, SLO violations (>500ms), and service degradation.

1.1 Proposed Proactive Scheduling Architecture

This project proposes an end-to-end autonomous proactive scheduling framework that eliminates reactive cold-start penalties by predicting workload intensity **5 minutes ahead** and preemptively binding pods to the optimal candidate node:

- Module 1 (Real Telemetry Collection):** Direct PromQL scraper capturing CPU, memory, network I/O, and HTTP request rates from Kubernetes worker nodes.
- Module 2 (Preprocessing & Sequence Building):** Chronological temporal split, robust MinMax scaling, lag feature generation (1-step, 2-step, 3-step lags, rolling window statistics), and 12-step lookback sequence tensors.
- Module 3 (LSTM-XGBoost Weighted Ensemble):** Combines the long-term temporal pattern extraction of an LSTM Recurrent Neural Network with the sharp non-linear gradient-boosted decision trees of XGBoost via convex weighted fusion ($R^2 = 0.9816$), lowest MSE/MAE).
- Module 4 (Multi-Objective Node Ranking Engine):** Evaluates candidate nodes across four weighted dimensions: Predicted CPU Headroom, Predicted Memory Headroom, Network RTT Latency, and Cluster Load Variance.
- Module 5 (Kubernetes Pod Scheduler & Binding):** Intercepts unscheduled pods matching `schedulerName: proactive-scheduler` and dispatches genuine Kubernetes API binding requests (`POST /api/v1/namespaces/{ns}/pods/{name}/binding`).
- Module 6 (Telemetry & Grafana Monitoring):** Real-time Prometheus metrics exporter and custom 14-panel Grafana cluster dashboard.

1.2 Key Benchmark Results (Publication Proof)

Metric	Reactive HPA Baseline	LSTM-Only Scheduler	Proposed Proactive Ensemble	Improvement
Average Latency	165.67 ms	94.58 ms	61.85 ms	-62.7%
P95 Latency (Tail)	734.99 ms	220.08 ms	111.18 ms	-84.8%
SLO Violations (>500ms)	10.33% (Breached)	0.00%	0.00% (Zero)	Eliminated
Cluster Load Variance	2.5056	0.7893	0.1491	16.8x Better
Model Forecast Accuracy (R^2)	N/A (No Forecast)	0.9620	0.9816	State-of-the-Art

2. Complete Technology Stack Breakdown

Layer / Component	Technologies Used	Technical Function in Project
Deep Learning	TensorFlow 2.18 / Keras, NumPy	2-layer LSTM Recurrent Neural Network capturing cyclical, long-term temporal sequence patterns.
Machine Learning	XGBoost Regressor, Scikit-Learn	Gradient-boosted decision trees for high-frequency non-linear sudden spike correction.
Mathematical Fusion	Convex Weighted Fusion ($\hat{y} = w_{\{1\}}\hat{y}_{\{1\}} + w_{\{2\}}\hat{y}_{\{2\}}$)	Combines predictions based on inverse-variance weights to minimize forecast error.
Container Orchestration	Kubernetes v1.29 (Pods, Nodes, Bindings)	Production container lifecycle management, scheduling framework, RBAC authorization.
Custom Scheduler	Go 1.21 Scheduler, Python Controller	Executes real <code>POST /api/v1/.../binding</code> Kubernetes API requests to bypass default scheduler.
Cloud Infrastructure	Google Cloud Platform (GCP), GKE, Cloud Run	Google Cloud SDK CLI 583.0.0, GKE Auth Plugin, GKE Standard 3-node cluster provisioning.
Telemetry & Metrics	Prometheus 2.45, PromQL, Grafana 10.2	Sub-second metric scraping, PromQL rate queries, 14-panel real-time operational dashboard.
Backend API	FastAPI, Uvicorn, Pydantic, Python 3.12	High-concurrency async REST API serving state, inference, node scoring, and cloud commands.
Frontend Portal	Vanilla HTML5, Modern CSS3, Chart.js	Cyber-dark responsive dashboard with 8 operational consoles and zero third-party UI framework bloat.
Dual-Mode Hosting	FastAPI Backend + Netlify Static Engine	Seamlessly switches between live local cluster backend and autonomous in-browser simulation.

2.1 Multi-Objective Mathematical Ranking Formulation

When a pod needs scheduling, the candidate nodes are filtered for hard constraints (Ready condition, schedulable, resource capacity). Surviving nodes are scored using the objective function:

$$S_i = w_{\text{cpu}}(1 - \hat{u}_i^{\text{cpu}}) + w_{\text{mem}}(1 - \hat{u}_i^{\text{mem}}) + w_{\text{lat}}(1 - \frac{L_i}{\text{SLO}}) + w_{\text{bal}}(1 - B_i)$$

Where $\hat{u}_i^{\text{cpu}}$ and $\hat{u}_i^{\text{mem}}$ are the **predicted future resource utilizations**, L_i is measured network latency against the 500ms SLO threshold, and $B_i = |\hat{u}_i^{\text{cpu}} - \bar{u}_{\text{cluster}}^{\text{cpu}}|$ is the cluster variance penalty.

3. Step-by-Step Live Demonstration Protocol

⚠ Presenter Setup Checklist Before Judges Arrive

- Ensure web server is running: open browser at `http://localhost:8000/` (or your Netlify deployment URL).
- Maximize browser window (F11 or full-screen) to present the clean cyber-dark dashboard.
- Check top header pills: `Models Loaded` , `SLO: <500ms` .
- Have Tab 1 (Cluster Topology) open as the starting screen.

Phase 1: Opening Hook & Cluster Topology (Tab 1)

Duration: 1.5 Minutes

What to show on screen: Tab 1 "Cluster Topology". Point to the live aggregate cluster CPU utilization, Memory utilization, Request rate, and the 3 candidate worker nodes (`worker-1` , `worker-2` , `worker-3`).

WHAT TO SAY TO THE JUDGES:

"Respected judges, in enterprise Kubernetes clusters, scheduling decisions are typically reactive—meaning pods are placed based only on current, lagging resource snapshots. In our project, we introduce **Workload-Forecasting Driven Proactive Scheduling**. On this dashboard, you can see our real-time cluster topology with 3 candidate nodes. Notice how node cards track both **Current CPU** and **Predicted CPU**. While `worker-1` is currently 65% loaded, our model predicts where cluster stress will emerge next."

Phase 2: Workload Forecasting Engine (Tab 2)

Duration: 2.0 Minutes

What to show on screen: Click Tab 2 "Workload Forecasting". Point to the real-time multi-series Chart.js graph. Drag the **LSTM Weight Slider** from 0.60 to 0.70 and back.

WHAT TO SAY TO THE JUDGES:

"Here in Tab 2 is our forecasting engine. Instead of relying on a single model, we implement a **Hybrid Weighted Ensemble**. The orange dashed line is our **LSTM Recurrent Neural Network**, which captures long-term cyclic patterns. The green dotted line is our **XGBoost Regressor**, which excels at capturing sharp gradient changes. The red solid line is our **Ensemble Fusion**. By combining them, our ensemble achieves an $\backslash(R^2\backslash)$ score of **0.9816**, outperforming both standalone models. Furthermore, administrators can dynamically adjust ensemble weights on the fly using these sliders."

Phase 3: Multi-Objective Node Ranking (Tab 3)

Duration: 2.0 Minutes

What to show on screen: Click Tab 3 "Multi-Objective Ranking". Show the table with the mathematical score breakdown (CPU Score, Memory Score, Latency Score, Balance Score, Final Composite Score). Drag the **Network Latency Weight Slider** higher.

WHAT TO SAY TO THE JUDGES:

"The default Kubernetes scheduler only looks at basic resource requests. Our scheduler uses a **Multi-Objective Composite Ranking Function** evaluated across four dimensions: CPU headroom, memory headroom, network RTT latency against our 500ms SLO, and cluster load balance variance. In this live table, `worker-2` is ranked #1 with a score of 0.762. Notice what happens when I increase the network latency weight—the node ranking re-sorts dynamically in real time, guaranteeing that high-latency nodes are never assigned latency-sensitive microservices."

Phase 4: Proactive Pod Scheduler Console (Tab 4)

Duration: 2.0 Minutes

What to show on screen: Click Tab 4 "Pod Scheduler Console". Point to the Live Decision Log Stream and the Active Pods Table. **Click the button: "Schedule New Pod (Proactive Binding)".**

WHAT TO SAY TO THE JUDGES:

"Now let us demonstrate the actual pod placement. In Tab 4, our scheduler controller intercepts unscheduled pods. When I click '**Schedule New Pod**', observe the log stream: the inference engine forecasts upcoming load, the node ranker identifies the optimal target node (`worker-2`), and our controller executes a real Kubernetes API binding call (`POST /api/v1/namespaces/proactive-system/pods/.../binding`). The new pod appears immediately in our active pod inventory without waiting for default scheduler latency."

Phase 5: Traffic Spike Simulator — The "Killer Demo" (Tab 5)

Duration: 2.5 Minutes

What to show on screen: Click Tab 5 "Traffic Spike Simulator". Point to the latency chart. **Click the red button: "Sudden Traffic Spike (Normal → 250 VUs)".**

WHAT TO SAY TO THE JUDGES:

"This is the definitive demonstration of our project's superiority over standard cloud solutions. When I trigger a **Sudden Traffic Spike**:

- The **red curve** represents standard **Reactive Kubernetes HPA**. Notice the severe latency spike reaching **720 ms**, severely breaching our 500 ms SLO line. Why? Because HPA only reacts after the spike arrives, incurring container cold-start delay.
- The **green curve** represents our **Proposed Proactive Scheduler**. Because our LSTM-XGBoost ensemble predicted the surge 5 minutes in advance, pods were scheduled preemptively. Latency stays completely flat below **110 ms** with **zero SLO violations**."

Phase 6: Empirical Baselines & Publication Plots (Tab 6 & 7)

Duration: 1.5 Minutes

What to show on screen: Click Tab 6 "Benchmarks & Baselines", then Tab 7 "Publication Plots". Highlight the 13 publication figures (Actual vs LSTM, Actual vs XGBoost, Actual vs Ensemble, MSE/RMSE/MAE comparisons, SLO violation charts).

WHAT TO SAY TO THE JUDGES:

"In Tab 6, our comparative experimental benchmarks demonstrate an **84.8% reduction in P95 tail latency** and complete elimination of SLO violations compared to HPA. In Tab 7, we provide 13 publication-grade Matplotlib figures at 300 DPI generated directly from our test runs, validating our methodology across predictive accuracy, load balance, and request latency."

Phase 7: Google Cloud Platform & GKE Cloud Integration (Tab 8)

Duration: 2.0 Minutes

What to show on screen: Click Tab 8 "Cloud Connector (GKE)". Show the Google Cloud SDK 583.0.0 status, GKE Auth Plugin Ready, and the Interactive Cloud Shell. Click "**kubectl get nodes**", "**kubectl get pods -A**", and "**Deploy to GKE**".

WHAT TO SAY TO THE JUDGES:

"Finally, our system is entirely cloud-native and production ready. Tab 8 provides direct integration with Google Cloud Platform and Google Kubernetes Engine (GKE). When I click '**Deploy to GKE**', it applies our Kubernetes manifests—deploying the namespace, RBAC permissions, and microservice workloads. Through our embedded **Interactive Cloud**

Shell, we can run live `kubectl` and `gcloud` commands directly against our cluster. The application is also packaged for instant zero-error hosting on Netlify."

4. Anticipated Judge Questions & Bulletproof Defense

Q1: Why did you use an ensemble of LSTM and XGBoost instead of just using LSTM?

Answer: LSTMs are recurrent neural networks with gated memory cells that excel at capturing non-linear cyclic temporal dependencies and long-term diurnal seasonality. However, LSTMs can experience lag during sudden, sharp gradient shifts and have high training overhead. Conversely, XGBoost (Extreme Gradient Boosting) is an ensemble of decision trees that excels at rapid split decisions and non-linear feature interactions without memory overhead. By combining them via convex weighted fusion ($w_1 \cdot \text{ext}\{\text{LSTM}\} + w_2 \cdot \text{ext}\{\text{XGBoost}\}$), our ensemble achieves the lowest MSE (0.0018) and the highest R^2 score (0.9816), compensating for each model's individual weaknesses.

Q2: How does your proactive scheduler bypass the default Kubernetes scheduler?

Answer: When a Kubernetes Deployment creates pods, the Pod specification includes `schedulerName: proactive-scheduler`. The default Kubernetes scheduler ignores any pod whose `schedulerName` does not match `default-scheduler`. Our custom scheduler controller monitors the Kubernetes API for pods matching our scheduler name that have `spec.nodeName == ""`. Once candidate nodes are ranked using our multi-objective score, our controller issues a Kubernetes API binding sub-resource call (`POST /api/v1/namespaces/{ns}/pods/{pod-name}/binding`), atomically setting the node name and placing the pod.

Q3: What happens if the machine learning model makes an incorrect prediction?

Answer: Our architecture incorporates safety fallbacks at two levels:

1. **Hard Constraint Filtering:** Prior to scoring, nodes must pass hard Kubernetes filters (Ready status, unschedulable flag, and allocatable capacity checks). An over-predicted or under-predicted score can never place a pod on an overloaded or failed node.
2. **Composite Balance Penalty:** The balance term $w_{\text{bal}}(1 - B_i)$ penalizes any single node from diverging significantly from the cluster mean utilization, preventing hot-spotting even under prediction noise.

Q4: Why does Reactive HPA suffer from high latency and SLO violations?

Answer: Reactive HPA calculates desired replicas as $\lceil \text{current} \cdot (\text{currentUtilization} / \text{targetUtilization}) \rceil$. It relies on delayed metrics scraping (default Prometheus scrape interval 15s to 30s) and requires pods to cross the threshold first. Once triggered, container creation, container image pulling, runtime initialization, and readiness probes introduce a 30 to 90 second delay. During sudden traffic spikes, all incoming requests queue up against the unscaled instances, pushing response latency over 700ms and violating our 500ms SLO limit.

Q5: How does this scale to a 1,000-node cluster in production?

Answer: In a large-scale cluster:

1. Inference runs asynchronously in background polling threads decoupled from the scheduling loop.
2. The node ranking engine operates in $O(N \log N)$ time where N is candidate nodes, and can use node filtering predicates (like Kubernetes node affinity and zone topology) to evaluate only surviving candidates.
3. We also provide a compiled Go Scheduler Plugin compatible with the native Kubernetes Scheduling Framework (`PreFilter`, `Filter`, `Score`, `Bind` extension points), running with sub-millisecond execution overhead.

5. Concluding Statement to the Judges

"In conclusion, our project demonstrates that reactive autoscaling in modern cloud architectures is fundamentally inadequate for latency-critical workloads. By uniting deep learning time-series forecasting, multi-objective ranking, and native Kubernetes API scheduling, we have achieved an 84.8% tail latency reduction, zero SLO breaches, and a 16.8x improvement in cluster

balance. The entire platform is deployed, verified, and ready for production cloud environments. Thank you, and we welcome your questions."